

Applying the Genetic Approach to Simulated Annealing in Solving Some NP-Hard Problems

Feng-Tse Lin, Cheng-Yan Kao, and Ching-Chi Hsu

Abstract—This paper presents a new stochastic approach called the annealing-genetic algorithm for solving some well-known combinatorial optimization problems. This approach incorporates genetic algorithms into simulated annealing to improve the performance of simulated annealing. Our approach has the following features: 1) it can be viewed as a simulated annealing algorithm with the population-based state transition and with the genetic-operator-based quasi-equilibrium control and 2) it can be viewed as a genetic algorithm with the Boltzmann-type selection operator. The goals of efficiency in our algorithm are 1) the gap between final solution and the optimal solution should be around 3% or less and 2) the computation time should be bounded by a polynomial function of the problem size. Empirically, the error rate of the proposed annealing-genetic algorithm for solving the multiconstraint zero-one knapsack problem is less than 1 percent, for solving the set partitioning problem it is less than 0.1 percent, and for solving the traveling salesman problem it is around 3 percent. In all the test cases, the annealing-genetic approach obtained much better performance than simulated annealing did. The time complexity of the proposed algorithm is empirically $O(n^2)$ for all the three problems.

A INTRODUCTION

MANY combinatorial optimization problems are intrinsically NP-hard [4], [19] and their deterministic polynomial time algorithms are very unlikely to exist. Heuristic approaches for these NP-hard problems have been the focus of research in this field for quite some time [19]. However, there are researchers who are trying some different nature-based stochastic approaches for approximately solving these NP-hard problems. Both simulated annealing and genetic algorithms are nature-based stochastic computational techniques; the former is based on thermodynamics and the latter is based on natural evolution. The major advantages of these nature-based algorithms are their broad applicability, flexibility, ease of implementation, and the potential of finding near-optimal solutions [6], [12].

Manuscript received November 15, 1991; revised November 6, 1992.

F.-T. Lin was with the Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan. He is now with the Department of Applied Mathematics, Chinese Culture University, Yangminshan, Taipei, Taiwan.

C.-Y. Kao and C. C.-Hsu are with the Department of Computer Science and Information Engineering, National Taiwan University, Taipei, Taiwan.

IEEE Log Number 9207502.

Kirkpatrick *et al.* [11] were the first to propose and demonstrate how to apply the simulation techniques from statistical physics to solve the combinatorial optimization problems. In simulated annealing, a control parameter called temperature is used to control the minimization search, which may occasionally move uphill. The mean and the variance of the cost function are decreasing during the course of the search process. Theoretically, the simulated annealing can be viewed as an algorithm that generates a sequence of Markov chains for a sequence of decreasing temperature values [1], [12]. At each temperature, the generation process is repeated again and again until the probability distribution of the system states approaches the Boltzmann distribution. If the temperature is decreased slowly enough, the Boltzmann distribution tends to converge to a uniform distribution on the set of globally minimal states. The analysis of simulated annealing based on time-homogeneous and time-inhomogeneous infinite length Markov chains has been carried out independently in the literature [5], [9], [21], [23]. However, in any implementation of the algorithm the Markov chain is of finite length. Therefore, asymptotic convergence can only be approximated. Due to these approximations, the simulated annealing algorithm is no longer guaranteed to find a global minimum with probability 1.

The performance analysis of an approximation algorithm should be considered with the quality of the final solution obtained as well as the execution time required by the algorithm. Thus, in recent years [12] many researchers have been engaged in designing an efficient annealing schedule or designing a parallel simulated annealing algorithm for finding a near-optimal solution around a predefined error rate and within a reasonable computation time. Unfortunately, as indicated by Aarts and Korst [1], not every annealing schedule reported in the literature turns out to be both efficient and certain of quality final results. Therefore, although the simulated annealing is theoretically guaranteed to converge to global optimums with probability 1 if the Markov chains are of infinite length, in practice, efficient annealing schedules are difficult to design.

Genetic algorithms were pioneered by Holland [8] and have been proved useful in a variety of search and opti-

mization problems over the years. Genetic algorithms are based on the survival-of-the-fitness principle, which tries to retain more genetic information from generation to generation. A genetic algorithm is composed of a reproductive plan that provides an organizational framework for representing the pool of genotypes of a generation. After the successful genotypes are selected from the last generation, the set of genetic operators such as crossover, mutation, and inversion is used in creating the offspring of the next generation. Whenever some individuals exhibit better than average performance, the genetic information of these individuals will be reproduced more often. Genetic algorithms work with a rich database of populations and simultaneously climb many peaks in parallel during the search so that the probability of trapping into a local minimum is reduced significantly. Thus, genetic algorithms are advantageous over simulated annealing for 1) direct manipulation of a coding, 2) search from a population, 3) search using stochastic operators, 4) blindness to auxiliary information, and 5) robustness [2], [6].

From our previous empirical results [13], [14], there are several observations from the performance analysis of the simulated annealing algorithm.

- 1) There is a tradeoff between the quality of the final solution obtained and the execution time required by simulated annealing, and the execution time is sensitive to the decrement ratio of the temperature.
- 2) It is easily trapped to local minima if the temperature drops too sharply.
- 3) It is not a trivial task for detecting the equilibrium of the system at each temperature, so that the length of the Markov chain may not be easily controlled.

-
1. Initialize the parameters of the annealing schedule;
 2. Randomly generate an initial state as the current_state;
 3. $k := 1$;
 4. **repeat**
 5. **repeat**
 6. call move generation strategy for generating the next_state;
 7. $\Delta C = \text{cost of the next_state} - \text{cost of the current_state}$;
 8. $P_r = \min \{1, \exp(-\Delta C/T_k)\}$;
 9. if $P_r > \text{random}[0,1]$ **then** the current_state := the next_state;
 10. **until** system equilibrium at T_k
 11. $T_{k+1} := T_k * \alpha$;
 12. **until** system has been frozen
 13. print out the current_state as the final state.
-

- 4) The initial value of temperature affects the total number of iterations required by the annealing.
- 5) There are still some chances to depart from good solutions if the number of iterations at low temperature regions are not large enough.

Subsequently, our work is motivated by these observations and by the idea of adapting the advantages of genetic algorithms to overcome the deficiencies of simulated annealing. However, in contrast to Davis and Ritter's [3] line of work, we do not optimize the parameters of sim-

ulated annealing via a number of experiments prior to the work of the annealing processes. Instead, we tackle the problem by designing an efficient annealing schedule in which the parameters are optimized dynamically during the course of the annealing processes. As a consequence, one obtains considerable flexibility in designing the annealing schedule to various NP-hard problems, not only to enhance the potential of finding near-optimal solutions but to reduce the running time of the algorithm. In this paper we propose an annealing-genetic approach incorporating the advantages of genetic algorithms into simulated annealing, so that it is very similar to a combination of steepest descent and Newton's method to improve each other for the constrained optimization problem of the continuous type [18]. We have devised a very efficient scheme for designing the annealing schedules to solve some well-known combinatorial optimization problems. The genetic approach to simulated annealing seems to facilitate the exhaustive and parallel treatment of the problem and to increase the probability of finding global minimums.

In the remainder of this paper we define the major problems of simulated annealing and discuss the characteristics of genetic algorithms in Section II. The proposed annealing-genetic approach is described in Section III. The experimental results and the performance analysis of this approach are reported in Section IV and Section V consists of discussions. Finally, the conclusions are given in Section VI.

II. SIMULATED ANNEALING AND GENETIC ALGORITHMS

Consider the usual outline of a simulated annealing (SA) algorithm shown as follows.

By the above SA algorithm, a finite-length Markov chain is generated at a certain temperature T_k with a single starting point (state) and a single ending point in a large search space Ω . The cost $C(i)$ of the ending point i is either lower or higher than the cost of the starting point, which should depend on how much computation time has been spent at that temperature T_k for generating the Markov chain. Thus, the Boltzmann distribution

$$\pi_i = \frac{e^{-C(i)/T_k}}{\sum_{j \in \Omega} e^{-C(j)/T_k}}, \quad i \in \Omega$$

where π_i is the probability of staying at point i , states that a higher probability of obtaining lower cost of the ending point occurs only after sufficient iterations at temperature T_k . Let a finite state, homogeneous Markov chain be a sequence $\{X_m | m = 1, 2, \dots\}$ of random variables that take values in $\Omega = \{1, 2, \dots, n\}$. It has been proved that if the generated Markov chains are under the conditions of being irreducible, aperiodic, and infinite lengths are allowed [12], [21], SA will find a global optimal solution with probability 1. Let Ω^* be the set of indexes of global minimal states in Ω . Thus, as T_k tends to zero while k tends to infinity, then

$$\lim_{T \downarrow 0, T_k = T} \lim_{m \rightarrow \infty} [\Pr(X_m \in \Omega^*)] = \lim_{T \downarrow 0} \sum_{i \in \Omega^*} \pi_i = 1.$$

The need of a very long Markov chain at each temperature is the reason that SA may require a long computation time for finding an optimal or a near-optimal solution.

Next, we study genetic algorithms (GA's) by applying them to solve several typical problems and find some clues for overcoming the above deficiency of SA. Consider the following outline of a GA:

1. Initialize the parameters of the genetic algorithm;
2. Randomly generate the old_population;
3. **for** generation := 1 **to** max_generation
4. Clear the new_population;
5. Compute the fitness of each individual in the old_population;
6. Copy the highest fitness of individual to the solution_vector;
7. **while** the no_of_individual < population_size **do**
8. Select two parents from the old_population based on their fitness values;
9. Perform the crossover of the parents to produce two offsprings;
10. Mutate each offspring based on mutation_rate;
11. Place the offspring to new_population;
12. **endwhile**
13. Replace the old_population by the new_population;
14. **endfor**
15. Print out the solution_vector as the final solution.

The above GA simulates an evolutionary process with n individuals (n points) in a large search space Ω . From the viewpoint of engineering, GA is an iterative process where each iteration has two steps, the evaluation step and the generation step. In the evaluation step, knowledge of domain is used to determine the fitness, which is a measure of the quality of a candidate. An evaluation function maps candidate solutions into the nonnegative real numbers. The generation step includes a selection operator and modification operators. The selection operator chooses individuals with a probability that corresponds to the relative fitness. Two chosen individuals, called the parents, produce children using the crossover, a genetic operator. The crossover operator exchanges a substring of the codes of the parents at the same randomly determined point or points. However, it does not create any new genetic material in the knowledge base. The mutation operator, on the other hand, randomly changes a bit in the structure introducing a new material into the knowledge base. From

another point of view, the mutation operator acts as a local search close to the current point in the search space while the crossover operator causes larger jumps in the search space. Finally, the descendents replace some individuals in the population after the generation step is complete.

We summarize several key features of GAs as follows:

- 1) GA's generate a sequence of populations of candidate solutions to the underlying optimization problem by employing a set of stochastic state transition genetic operators to transform each population of candidate solutions into a descendent population.
- 2) GA's work from a population instead of a single state. By maintaining a population of well-adapted states, the probability of trapping into a local minimum is greatly reduced.
- 3) The crossover operation tries to retain genetic information from generation to generation, assuming that the genetic information always contains important substructures of the quality solutions. The av-

erage performance of the next generation is indeed better than the previous.

- 4) Although the mutation operation has the effect of destroying the structure of a solution, it does not always produce an undesirable solution; there is still a chance for producing a better one.
- 5) Selecting parents based on their fitness values means that parents with a higher value always have a higher probability of contributing one or more offspring in the next generation. In contrast, parents with a lower fitness value still have a chance to reproduce. Thus, the probability of escaping from local minima of GA's is higher than that of SA.
- 6) The genetic operators that emulate biological system behavior possess the property that the conditional dependence of each population is completely determined by its predecessor population. Then, the sequence of populations evolves as a Markov chain.

- 7) If the average cost of the population is decreased from generation to generation, one can assure a faster convergence ratio of GA's.

III. THE PROPOSED ANNEALING-GENETIC APPROACH

Since our goal is to look for an approach to design an efficient annealing schedule, a reasonable computation time and a near optimal solution are essential. The goals of efficiency in our approach are 1) the gap between the final solution and the optimal solution should be around 3% or less and 2) the computation time must be bounded by a polynomial function of the problem size. We start with the following key issues affecting the efficiency of SA from our previous empirical results [13], [14]. These key issues are critical factors to be solved in our work:

- 1) what is the best range of the initial value of the temperature;
- 2) how to determine the total length of the Markov chain at each temperature;
- 3) how to detect the equilibrium condition of the system at each temperature;
- 4) how to keep the system from trapping into a local minimum;
- 5) how to know the system has been frozen.

We tackle these key issues of SA by designing an annealing schedule that has incorporated the advantages of GA's. The important parameters affecting the efficiency of SA are the initial temperature, the total length of the Markov chain at each temperature, the equilibrium condition, and the frozen condition. We use the notions from GA's to initiate and control these parameters. Then, we call this strategy the annealing-genetic (AG) approach for improving SA.

A. Initial Value of Temperature

The implementation of SA is mostly based on the simulation of condensed matter physics. The initial temperature is set to a sufficiently high value, resulting in a high degree of randomness in the initial stages of the search [9], [12], [23]. Thus, this adequately hot condition will produce many redundant iterations in the course of SA. Such a strategy is not used in our proposed AG approach. Instead, we use the concepts of GA's. The AG algorithm starts with a randomly generated population P_0 . Next, the genetic operators are applied to produce a new population P'_0 by rejecting the higher cost offspring so that the average cost must be less than the random cost P_0 . Then, the Markov chains are generated from P'_0 by a small value of temperature until the next generation P_1 is created. That is, starting with a random point (i.e., individual or state) from P'_0 , the next point is generated from the current point by the move generation strategy. If it is accepted by the Metropolis criterion, the next point not only becomes the current point but also a member of P_1 ; otherwise, a new starting point from P'_0 is attempted. The process is continued until P_1 is generated. The initial value of temper-

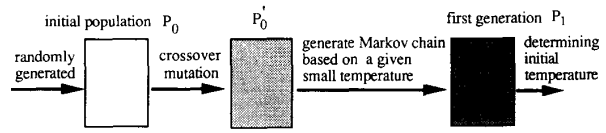


Fig. 1. Determining the initial value of temperature.

ature is to be determined in the following manner. For the sake of the efficiency of the algorithm, we define the acceptance probability of detrimental move to be 0.6. From the Metropolis criterion $Pr = \exp(-\Delta C/T)$, where $\Delta C = \text{new cost} - \text{old cost}$, we obtain $T = -\Delta C / \ln Pr = -\Delta C / \ln 0.6 \cong 2\Delta C$. The largest detrimental move in this generation is $\Delta C = (\text{the highest cost} - \text{the lowest cost}) / \text{population_size}$. Thus, the initial temperature value is obtained by dividing the difference between the points of highest cost and the lowest cost P_1 by half of the population size, i.e.,

$$\text{initial temp} = \frac{\text{the highest cost} - \text{the lowest cost}}{\text{population_size} / 2}.$$

Finally, we let the lowest cost of P_1 be the starting point and the annealing process is then initiated (see Fig. 1).

B. The Total Length of the Markov Chain

The total length of the Markov chain at temperature T_k is bounded by the population size and the subchains are generated from multiple points of the population. Again, starting with the point of the lowest cost of population P_k , the next point is generated from the current point by the move generation strategy. If it is accepted by the **Metropolis criterion**, the next point not only becomes the current point but also a point of the next generation P'_{k+1} ; otherwise, a new starting point from P_k is attempted. The generated points in P'_{k+1} are the candidates for the population of the next generation P_{k+1} .

C. The Quasi-Equilibrium of the System

After the population P'_{k+1} has been generated at each temperature T_k , the genetic operators such as crossover, mutation, and inversion are applied to produce a new population P_{k+1} of the next generation. A detailed discussion of these genetic operators is given in Section V. The parents are selected based on their fitness values and their offspring must have their costs less than the average cost of the last generation; otherwise, the parents are reproduced in this generation. The mutation of the offspring is based on the predefined mutation rate, which is always a very small value. From our point of view, the new generation P_{k+1} denotes the quasi-equilibrium of the system at T_k (see Fig. 2). The main difference between SA and the proposed AG approach is shown in the following figures. Fig. 3 shows a homogeneous Markov chain with a single starting point and a single ending point by SA at a certain temperature. The cost of the ending point may be lower or higher than the cost of starting point, depending on how much time has been spent at that temperature. The

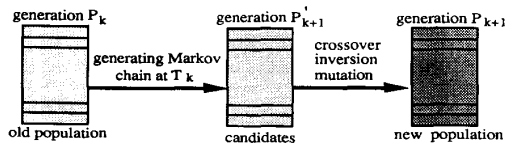
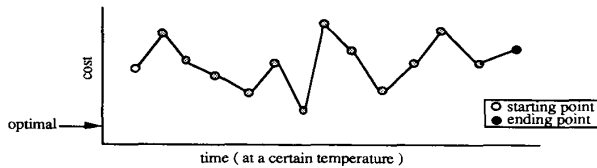
Fig. 2. Markov chains at temperature T_k and its quasi-equilibrium.

Fig. 3. A finite-length Markov chain is generated at a certain temperature with a single starting point and a single ending point by SA.

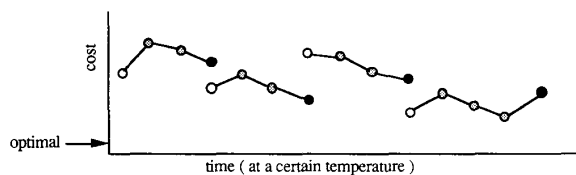


Fig. 4. At a given temperature there are many starting points and ending points to construct Markov chains by the AG approach.

Boltzmann distribution states that at any given temperature the probability of obtaining a lower cost ending point is higher when the iterations are sufficiently large. However, in our approach as shown in Fig. 4, there are many starting points and many lower cost ending points that are part of the Markov chains at a given temperature. After the crossover operations, we obtain the new ending points with costs lower than those of old ending points. Thus, at each temperature the Markov chains generated by the proposed AG approach are close to a monotonically decreasing function. (See Fig. 5.)

D. System Frozen

What is essential to the convergence proof for the system is the fact that the average cost of each generation is less than or equal to that of the last generation. The point of lowest cost in each generation is copied to an independent solution vector as the search continues and is printed at the end of the run. Finally, when 80 percent of the population in a certain generation has the same cost as the solution vector, the frozen condition is signaled. After producing one more generation, the algorithm is terminated.

The proposed AG approach consists of a two-stage cycle. The first stage is the search by the annealing and the second stage is the evolution by the genetic operators. Fig. 6 shows the basic concept of the AG approach and the concept is described in the following way. Starting off with a set of arbitrary nodes from the population of cur-

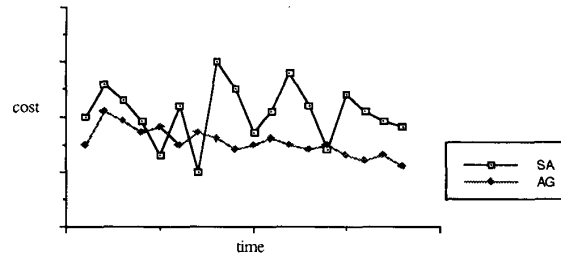


Fig. 5. The Markov chains generated by AG and SA.

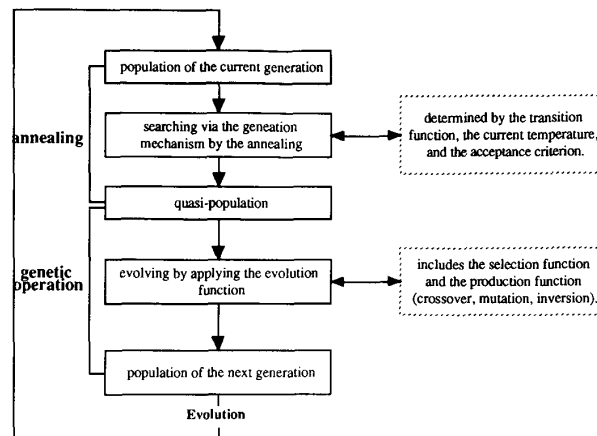


Fig. 6. The concept of the annealing-genetic approach.

rent generation and with a certain temperature, our approach creates a number of different search paths trying to find better solutions simultaneously. A heuristic method with a probability measurement is evaluated to guide these search paths when they climb in the search space. The heuristics includes a move generation mechanism and an acceptance criterion. At each temperature, the nodes visited by these search paths are joined to a set that is called a candidate population. The completion of the annealing stage is reached with the candidate population being created by the search paths. After completing a candidate population, the second stage begins. The genetic operation stage is an evaluation of the evolution function. The evolution function consists of selection and production functions (i.e., crossover, inversion, and mutation). After applying the evolution function to the candidate population, it creates the population of a generation. The next generation is more mature than the last one so that the average cost of the population is decreased. The new structures of the population and the new temperature are then evaluated, and the cycle repeats until the system has frozen. In our approach, the high-performance points become the building blocks for the structures of future generations and are propagated in parallel, generation after generation. The process to generate a sequence of populations from generation to generation is indeed an evolution.

The proposed genetic approach to simulated annealing is summarized as follows:

The annealing-genetic (AG) algorithm

1. Initialize the parameters, i.e., population_size, T_0 , and α ($0 < \alpha < 1$);
2. Randomly generating P'_0 ;
3. Apply genetic operators to P'_0 to create P_0 ;
4. Calculate the fitness and the cost for each point in P_0 ;
5. Calculate the average cost of P_0 ;
6. solution_vector := current_point := the lowest cost point in P_0 ;
7. $k := 0$;
8. **while** system is not frozen **do**
9. no_of_point := 0;
10. **while** no_of_point $\leq$ population_size **do**
11. Generate next_point from current_point by the move generation strategy;
12. $\Delta C :=$ cost of next_point - cost of current_point;
13. $Pr := \min [1, \exp (-\Delta C / T_k)]$; guaranteed that when next cost is lower it gets accepted
14. **if** $Pr > \text{random}[0,1)$ **then** put next_point into P'_{k+1} ;
15. current_point := next_point;
16. no_of_point := no_of_point + 1;
17. **else** pick another point from P_k as current_point;
18. **endwhile**
19. Apply the genetic operators to P'_{k+1} to create P_{k+1} ;
20. Calculate the fitness and the costs for each point in P_{k+1} ;
21. Calculate the average_cost of P_{k+1} ;
22. **if** the lowest cost point in $P_{k+1} <$ solution_vector **then** update solution_vector;
23. **if** it is the initial stage **then** determining the initial temperature T_1 ;
24. $T_1 := (\text{the highest cost} - \text{the lowest cost}) / (\text{population_size} / 2)$;
25. **else** $T_{k+1} = T_k * \alpha$;
26. current_point := the lowest cost point in P_{k+1} ;
27. $k := k + 1$;
28. **if** frozen condition is signaling **then** set system is frozen;
29. **endwhile**
30. Perform the local search procedure;
31. Print out the solution_vector as the final solution;

IV. EMPIRICAL RESULTS

This section describes some computational results with the proposed AG algorithm for three famous NP-hard problems. The first is the multiconstraint zero-one knapsack problem (MCKP).

A. The MCKP

The MCKP is a special case of the general zero-one integer programming and arises in the areas of capital budgeting as well as resource allocation [19]. The MCKP can be used as a subproblem for solving a large integer programming problem. The problem can be stated in the following way. We are given n objects and a knapsack. Object j has weight w_{ij} by the i th measure and the knapsack also has m capacities due to m different measures. That is, the capacity of the knapsack by the i th measure is b_i and the weight of object j by the i th measure is w_{ij} . If object j can be placed into the knapsack satisfying all different m measures, then a profit of $p_j x_j$ is earned. The objective is to obtain a filling of the knapsack that maximizes the total profit earned. Since the knapsack capacity is limited, we require the total weight of all chosen ob-

jects to be at most b_i by each different measure i , $i = 1, \dots, m$. Formally, this problem can be formulated as follows:

$$\text{maximize } z = \sum_{j=1}^n p_j x_j$$

$$\text{subject to } \sum_{j=1}^n w_{ij} x_j \leq b_i \text{ and } x_j = 0 \text{ or } 1$$

$$\text{where } i = 1, \dots, m \text{ and } j = 1, \dots, n.$$

Without a loss of generality, all p_j , w_{ij} , and b_i are assumed to be nonnegative integers.

For the first set of experiments, ten test problems with ten constraints (i.e., $m = 10$) and a problem size of $n = 10, 20, 30, \dots, 100$ are randomly generated. The w_{ij} are uniformly distributed random numbers from 0 to 500 and each p_j is equal to each w_{1j} . By letting x_j randomly equal 0 or 1 and defining

$$b_1 = \sum_{j=1}^n w_{1j} x_j$$

TABLE I
RESULTS OF THE FIRST SET OF 10 RANDOMLY GENERATED DATA ON THE MCKP

(1) $n * m$	AG						SA				
	Cost			(2) Legal Moves	(3) Time (s)	$\sqrt{(2)}$ (1)	$\sqrt{(3)}$ (1)	Cost			Time (s)
	Optimal	Final	Gap (percent)					Final	Gap (percent)	Legal Moves	
10 * 10	1922	1922	0.00	246	2	0.157	0.014	1922	0.00	698	3
20 * 10	3485	3485	0.00	731	6	0.135	0.012	3485	0.00	1446	6
30 * 10	3779	3779	0.00	878	8	0.099	0.009	3779	0.00	2150	11
40 * 10	3921	3921	0.00	753	9	0.069	0.008	3921	0.00	2916	19
50 * 10	6879	6879	0.00	1129	16	0.067	0.008	6878	0.01	3601	28
60 * 10	7913	7913	0.00	1377	22	0.062	0.008	7911	0.02	4321	37
70 * 10	9141	9140	0.01	1201	22	0.050	0.007	9138	0.03	5042	49
80 * 10	9482	9481	0.01	1352	29	0.046	0.007	9478	0.04	5767	62
90 * 10	14970	14968	0.01	1395	33	0.042	0.006	14964	0.04	6501	78
100 * 10	19060	19060	0.00	1787	49	0.042	0.007	19052	0.04	7231	91

1) n = problem size, m = number of constraints.

2) The time in seconds is the turnaround time. The algorithm is implemented on a 80386-based PC written in Turbo-C.

3) The annealing schedule for SA is as follows: initial iterations = $2 * n$, upper bound of iterations = $10 * n$; increment ratio of iterations = 1.2, cooling ratio = 0.85, initial temperature = 10.0 (with sufficiently hot conditions), frozen temperature = 1.0, and stopping criterion = no better solution obtained.

4) The move generation strategy is stated in Section V.

the optimal solution of each problem is determined. Therefore, the optimal solution of this set of problems is known and the performance of the algorithm can be easily determined. Table I shows the performance analysis of AG and SA on this set of 10 randomly generated problems where AG obtains better performance than SA. Although the final solutions obtained by both methods are very close to the optimal solutions, SA always needs more iterations as well as running time, about twice, than that for AG (see Fig. 7). Furthermore, the solution quality of AG is better than that of SA. Fig. 8 shows the performance analysis of AG where the square root of the total legal moves divided by the product of a problem size n and constraints m is decreased as n grows, and the square root of the turnaround time divided by $n * m$ is gradually stable as n grows. This implies that the algorithm is empirically better than $O(n^2)$.

The second set of experiments consists of well-known test problems from Peterson and Senju-Toyoda [20], [22]. The comparative results of AG and SA are shown in Table II. The final solutions of these problems by AG are always less than 0.8 percent error rate of the optimal solution. Fig. 9 shows the comparison of SA and AG in regard to the ratio of the square root of legal moves divided by the problem size where AG has fewer legal moves than SA. The legal moves are the lowest number of iterations used in the algorithm in searching for better solutions. Fig. 10 shows the comparison of SA and AG in terms of the ratio of the square root of the running time divided by the problem size in which AG has a shorter running time than SA. Empirically, the proposed annealing-genetic algorithm is $O(n^2)$ for MCKP.

B. The Set Partitioning Problem

The second problem is concerned with the set partitioning problem. The set partitioning problem is formally de-

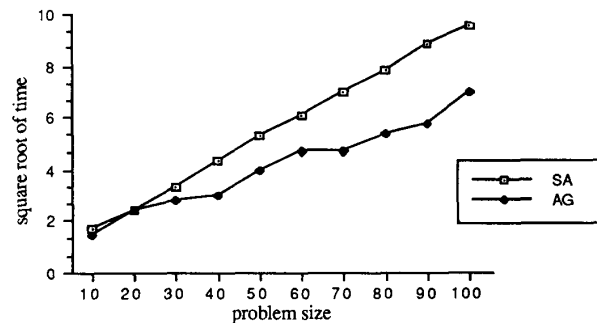


Fig. 7. Comparison of SA and AG in regard to the square root of the running time.

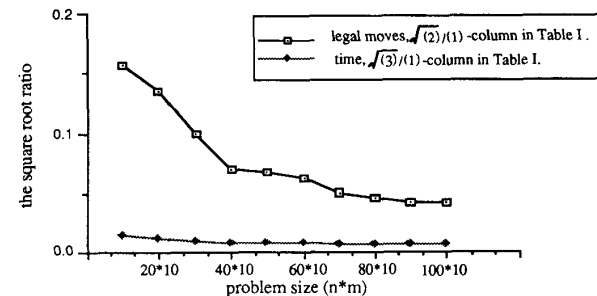


Fig. 8. Performance analysis of AG on the ten randomly generated data of MCKP.

fined as follows [4]. Given a finite set A and a size $s(a) \in \mathbb{Z}^+$ for each $a \in A$, we want to find a subset $A' \subseteq A$ such that

$$\sum_{a \in A'} s(a) = \sum_{a \in A - A'} s(a).$$

TABLE II
RESULTS OF THE SECOND SET OF WELL-KNOWN TEST PROBLEMS ON THE MCKP

(1) $n * m$	AG						SA				
	Cost			(2) Legal Moves	(3) Time (s)	$\sqrt{(2)}$ (1)	$\sqrt{(3)}$ (1)	Cost			
	Optimal	Final	Gap (percent)					Final	Gap (percent)	Legal Moves	Time (s)
6 * 10	3800	3800	0.00	419	3	0.341	0.029	3800	0.00	214	2
10 * 10	8705	8705	0.00	690	5	0.263	0.022	8335	4.25	911	6
15 * 10	4015	4015	0.00	811	7	0.190	0.018	4015	0.00	1473	9
20 * 10	6120	6120	0.00	1083	10	0.165	0.016	6120	0.00	2965	16
28 * 10	12400	12400	0.00	1517	17	0.139	0.015	12330	0.56	4097	23
39 * 5	10618	10556	0.58	1848	28	0.232	0.027	10552	0.62	8649	45
50 * 5	16537	16469	0.41	3449	56	0.235	0.030	16382	0.94	17121	84
60 * 30	7772	7758	0.18	6239	178	0.044	0.007	7692	1.03	38586	263
60 * 30	8722	8656	0.76	3131	92	0.031	0.005	8596	1.44	18673	121

- 1) The problem sizes of 60 * 30 are from Senju and Toyoda [22] and the others are from Peterson [20].
- 2) The annealing schedule for SA is as follows: initial iterations = 2 * n , upper bound of iterations = 10 * n ; increment ratio of iterations = 1.2, cooling ratio = 0.7, initial temperature = $\max p_j - \min p_j$ (with sufficiently hot conditions), frozen temperature = 1.0, and stopping criterion = no better solution obtained.
- 4) The move generation strategy is stated in Section V.

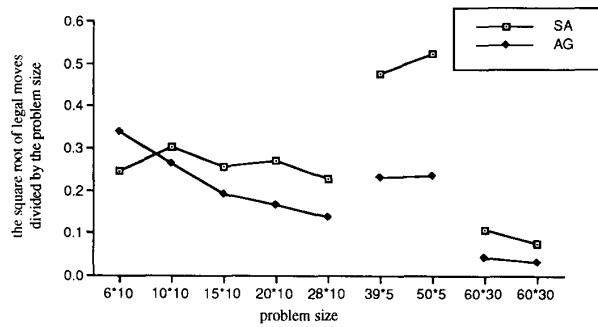


Fig. 9. Comparison of SA and AG in regard to the ratio of the square root of legal moves divided by the problem size.

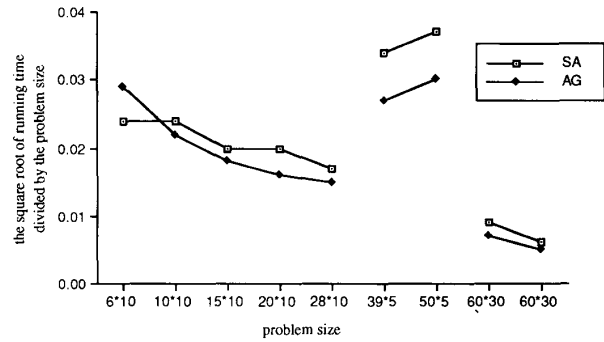


Fig. 10. Comparison of SA and AG in regard to the ratio of the square root of running time divided by the problem size.

Following Garey and Johnson [4] the set partitioning problem can be transformed to the MCKP as follows:

$$\begin{aligned}
 & \text{maximize } \sum_{j=1}^n w_j x_j \\
 & \text{subject to } \sum_{j=1}^n w_j x_j \leq \frac{\sum_{j=1}^n w_j}{2} \\
 & \text{where } w_j \geq 0 \text{ and } x_j = 0 \text{ or } 1.
 \end{aligned}$$

For the first set of experiments, the problem sizes of $n = 10$ to $n = 120$ are randomly generated with the w_{ij} from 0 to 499. Because of the 16-bit integer limitation, the w_{ij} of the larger problems of 130, 140, and 150 are randomly generated from 0 to 299. Table III is the performance analysis of AG and SA on this set of 15 randomly generated data. Again, although the solution quality obtained by both methods is about equal and very close to the optimal solutions, SA always need more iterations in addition to as much running time, about twice, than that of AG. Fig. 11 is the performance analysis of AG on this set

of randomly generated data. The ratio of the square root of the total moves divided by n is monotonically decreasing as n grows up. The square root of turnaround time divided by n is also gradually stable as n grows. The second set of experiments is concerned with $n = 100$ and consists of ten randomly generated test data. As shown in Table IV and Fig. 12, the ratio of the square root to the running time divided by n and the ratio of the square root of the legal moves divided by n are very stable for these test data, and the gaps between the final and the optimal solutions are less than 0.03 percent. In fact, for seven out of the 10 problems the optimal solutions are obtained. The proposed algorithm is empirically better than $O(n^2)$ for the set partitioning problem.

C. The Traveling Salesman Problem

The third problem is the traveling salesman problem. This problem is more difficult to solve than the other two problems because the solution space is much larger. The first set of experiments consist of 9 problems that have 16, 25, 36, 49, 64, 81, 100, 121, and 141 cities, respec-

TABLE III
RESULTS OF THE FIRST SET OF 15 RANDOMLY GENERATED DATA ON THE SET PARTITIONING PROBLEM

(1) n	Cost			(2) Legal Moves	(3) Time (s)	$\sqrt{(2)}$ (1)	$\sqrt{(3)}$ (1)	SA			
	Optimal	Final	Gap (percent)					Final Cost	Gap (percent)	Legal Moves	Time (s)
10	1202	1202	0.00	124	1	1.114	0.100	1200	0.00	82	0.5
20	2456	2456	0.00	259	2	0.805	0.071	2456	0.00	171	1
30	3644	3644	0.00	304	2	0.581	0.047	3643	0.03	497	2
40	4939	4939	0.00	376	3	0.485	0.043	4939	0.00	743	3
50	6480	6480	0.00	467	3	0.432	0.035	6480	0.00	836	4
60	7600	7599	0.01	548	5	0.390	0.037	7599	0.01	1105	5
70	8907	8906	0.01	598	5	0.349	0.032	8906	0.01	1376	6
80	10548	10547	0.00	713	7	0.334	0.033	10547	0.00	2070	8
90	11713	11712	0.00	762	9	0.307	0.033	11712	0.00	3628	14
100	12897	12897	0.00	880	10	0.297	0.032	12897	0.00	4921	19
110	14166	14166	0.00	947	12	0.280	0.031	14166	0.00	5452	21
120	15438	15438	0.00	996	14	0.263	0.031	15438	0.00	6776	25
130	10051	10051	0.00	1070	15	0.252	0.030	10051	0.00	7552	29
140	10766	10766	0.00	1215	18	0.249	0.030	10766	0.00	7552	29
150	11597	11596	0.00	1272	19	0.238	0.029	11596	0.00	9613	34

- 1) The annealing schedule for SA is as follows: iteration ratio = 1.2, initial iterations = n , upper bound of iterations = $(n/10) * n$; cooling ratio = 0.85, initial temperature = 5.0, frozen temperature = 0.1 (with sufficiently hot conditions), and stopping criterion = no better solution obtained.
2) The move generation strategy is stated in Section V.

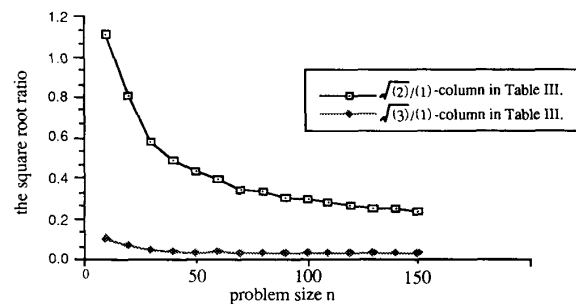


Fig. 11. Performance of AG for each problem size on the set partitioning problem.

TABLE IV
RESULTS OF THE SECOND SET OF RANDOMLY GENERATED DATA ON THE SET PARTITIONING PROBLEM BY AG

(1) $n = 100$	Cost			(2) Moves	(3) Time (Sec)	(2) (1)	(3) (1)	$\sqrt{(2)}$ (1)	$\sqrt{(3)}$ (1)
	Optimal	Final	Gap (percent)						
	2747	2747	0.00	901	10	9.01	0.10	0.300	0.032
	3697	3697	0.00	881	11	8.81	0.11	0.279	0.033
	5297	5297	0.00	892	11	8.92	0.11	0.299	0.033
	7897	7897	0.00	892	10	8.92	0.10	0.299	0.032
	7877	7877	0.00	899	10	8.99	0.10	0.299	0.032
	10792	10790	0.02	872	10	8.72	0.10	0.295	0.032
	11377	11377	0.00	899	9	8.99	0.09	0.299	0.030
	3541	3540	0.03	893	10	8.93	0.10	0.299	0.032
	8276	8276	0.00	883	10	8.83	0.10	0.297	0.032
	13279	13277	0.02	872	10	8.72	0.10	0.295	0.032

tively, with the cities being regularly placed in a square Euclidean space. This set of problems has the property that the optimal solutions can be easily determined because of the structures of the cities. The problems mentioned above are the ideal test cases of the properties of any Euclidean TSP algorithm. The initial population con-

sists of randomly generated tours. The population size is proportional to the number of cities. Table V shows the final results obtained by AG and by SA in each problem and the error rate of the gap in percentage to the optimal cost. As shown in Fig. 13, among all the test problems the largest error rate in AG is 3.1 percent, while in SA it

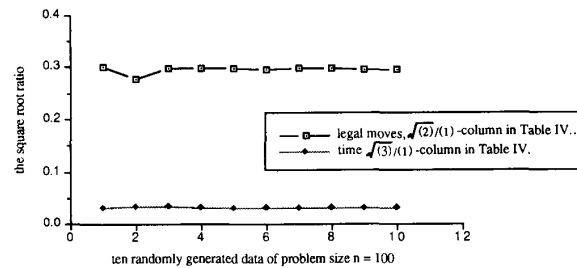


Fig. 12. Performance of AG on ten randomly generated data of the same problem size $n = 010$.

TABLE V
RESULTS OF THE REGULAR-GRID TSPs BY AG AND BY SA

City	Optimal Cost	AG			SA		
		Final Cost	Gap (percent)	Time (s)	Final Cost	Gap (percent)	Time (s)
16	16.00	16.00	0	8	16.00	0	36
25	25.41	25.41	0	19	26.24	3.3	62
36	26.00	36.00	0	39	37.66	4.6	101
49	49.41	49.41	0	66	51.07	3.4	170
64	64.00	64.83	1.3	128	67.07	4.8	283
81	81.41	83.90	3.1	212	84.89	4.3	397
100	100.0	102.54	2.5	394	108.21	8.2	596
121	121.41	126.38	3.1	568	130.21	7.3	846
144	144.00	147.31	2.3	792	154.45	7.2	1233

- 1) The execution time actually included the time spent in the display of the tours periodically.
- 2) The annealing schedule for SA is as follows: iteration ratio = 1.2, initial iterations = $2 * n$ (n is the number of cities), upper bound of iterations = $10 * n$; cooling ratio = 0.9, initial temperature = 10.0 (sufficiently hot enough conditions), frozen temperature = 0.05, and stopping criterion = no better solution obtained.
- 3) The move generation strategy is stated in Section V.

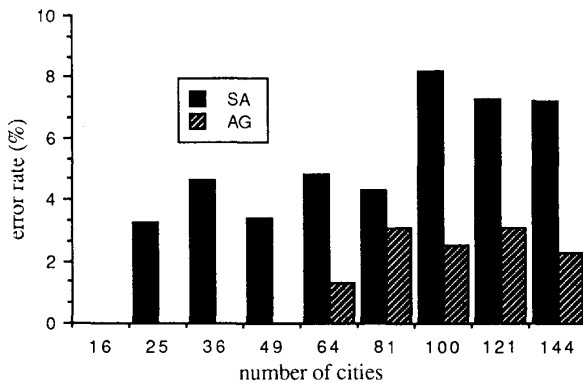


Fig. 13. Comparison of SA and AG in regard to the error rate to the optimal solution.

is 8.2 percent. Table VI is the performance analysis of AG on the regular-grid TSP's. Again, as shown in Fig. 14, both the ratios of the square root of the total number of moves and the square root of the turnaround time divided by the number of cities approach to a constant as the number of cities increase. Therefore, the proposed AG approach is also empirically $O(n^2)$ for the regular-grid TSP's. The initial cost of each problem is the cost of a random tour; it usually serves as a starting state in the

conventional annealing schedule. In our approach, as indicated in Fig. 15, the starting state of each problem has a lower cost that is generated by the initial stage of the algorithm.

In other set of experiments, three TSP's taken from the literature [24], 30-city, 50-city, and 75-city problems are used for performance analysis. The best-known result of the 30-city problem in the literature is 421. However, we have obtained the new optimal solution of 420 by the proposed AG approach. Although the final solutions in the 50-city and 75-city problems are not optimal, their gaps to the best-known solutions are not greater than 3 percent; the total number of moves are reduced significantly and the gaps to the optimal solutions are smaller compared to SA, as indicated in Table VII.

V. DISCUSSION

In this section we first discuss the efficiency of the AG algorithm by examining a given simple example of the MCKP. The AG approach has the feature whereby it can be viewed as 1) an SA algorithm with the population-based state transition and with the genetic-operator-based quasi-equilibrium control and 2) as a GA with a Boltzmann-type selection operator. The AG approach is a two-stage cycle: the first stage is an annealing stage by SA and the second stage is a genetic operation stage by GA. In each anneal-

TABLE VI
PERFORMANCE ANALYSIS OF AG ON THE REGULAR-GRID TSP'S

City	Initial Cost	Starting Cost	Moves	$\sqrt{\text{Moves}}/\text{City}$	Time (s)	$\sqrt{\text{Time}}/\text{City}$
16	26.06	23.77	976	1.85	8	0.177
25	65.98	39.42	2275	1.91	19	0.174
36	112.67	73.38	4698	1.90	39	0.173
49	168.78	123.58	8465	1.88	66	0.166
64	262.56	177.18	17125	2.04	128	0.177
81	351.44	309.03	34851	2.30	212	0.180
100	507.73	414.84	63926	2.53	394	0.198
121	690.50	533.54	91637	2.50	568	0.197
144	905.65	716.05	122344	2.42	792	0.195

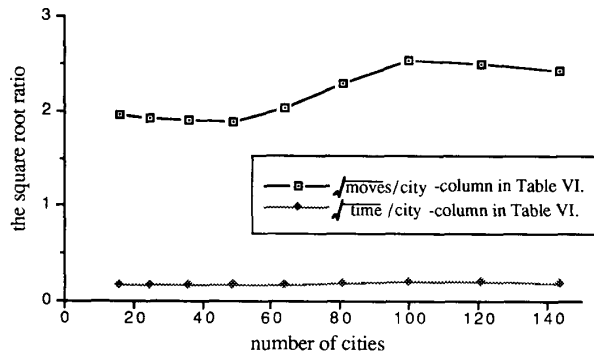


Fig. 14. Performance of AG in regard to the ratio of the square root ratio of moves and time for each problem size on TSP.

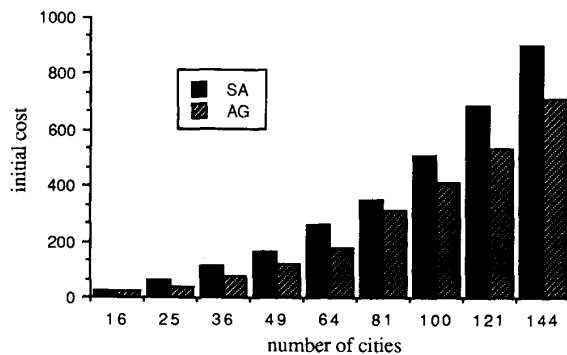


Fig. 15. The cost of initial state for each problem in AG and SA.

ing stage, an iterative generate-and-test procedure is executed to generate many search paths that create a candidate population. In each genetic operation stage, a set of genetic operators is applied to the candidate population to evaluate the cost function of each member and to produce the population of the next generation. Usually, the genetic operations make the next generation more mature than the last generation and manipulate a set of schemata of the population. A schema is a building block describing a subset of points with similarities at certain positions in the bit-string (a representation in the points) [6], [8]. The schemata are also used to guide the searches of the annealing stage in each cycle. They have the power of guid-

ing the directions of the search paths toward high-performance regions. Therefore, the concept of schemata is a powerful tool for a genetic approach to simulated annealing. The schema structure restricts the search in a hyperplane and inherits the hyperplane subdivision of the search space defined by the schemata. Because schemata are associated with a characteristic fitness, it follows that the hyperplane subdivision also have a characteristic fitness. The relationship between the whole search space and the high-performance subspace is exactly the relationship we want to establish. Another important feature of the genetic approach is the reliance on global information, i.e., the schemata that are being processed and consequently emerge to dominate the population spread over the entire space. The underlying concept of schemata is that schemata are a *degenerate* description of the entire space that confines the searches to the high-performance regions. Thus, the search paths will eventually converge to an optimal or near-optimal point.

A. The Schemata Theory

In the following, we discuss why the schemata exhibit high efficiency in search space to guide the searches of the search paths. We give a simple instance of the MCKP with the problem size $n = 6$ and the constraints $m = 10$, which is shown in Table VIII. The optimal solution is a vector $X = [0, 1, 1, 0, 0, 1]$ with cost = 3800. By examining the fitness of any one point, one might expect to obtain information about other points that have a similar structures. Usually, we can associate 3^n schemata over the extended alphabet $\{0, 1, *\}$ with length = n of a point. The notion of schemata arises from the observation that in evaluating the fitness of a point we can also derive implicit knowledge about the schemata that describe that point. The accuracy of this extrapolation depends on the specification of the given schema. It is important to identify the schemata that have above-average fitness in the population, because the important similarities among these highly fit points can help guide a search during the annealing stage by SA. These high-performance hyperplane schemata also play an important role in investigating the structure of large, complex search spaces. However, the high-performance schemata are limited in each hyperplane. Table IX shows the high-performance schemata in

TABLE VII
PERFORMANCE ANALYSIS OF SA AND AG ON THE THREE TSP'S FROM [24]

Source	Cities	Best-Known	SA			AG		
			Final	Gap (%)	Moves	Final	Gap (%)	Moves
Oliver	30	421	424	1	24617	420 ^a	0	12620
Eilon	50	428	443	3.7	68512	436	1.9	28111
Eilon	75	545	580	6.4	173250	561	3	95506

^a The best-known solution obtained by AG.

1) The annealing schedule for SA is as follows: iteration ratio = 1.2, initial iterations = $2 * n$ (n is the number of cities), upper bound of iterations = $10 * n$; cooling ratio = 0.9, initial temperature = 10.0 (with sufficiently hot enough conditions), frozen temperature = 0.05, and stopping criterion = no better solution obtained.

2) The move generation strategy is stated in Section V.

TABLE VIII
TEST DATA WITH $n = 6$ AND $m = 10$

j	p_j	w_{1j}	w_{2j}	w_{3j}	w_{4j}	w_{5j}	w_{6j}	w_{7j}	w_{8j}	w_{9j}	w_{10j}
1	100	8	8	3	5	5	5	0	3	3	3
2	600	12	12	6	10	13	13	0	0	2	2
3	1200	13	13	4	8	8	8	0	4	4	4
4	2400	64	75	18	32	42	48	0	0	0	8
5	500	22	22	6	6	6	6	8	8	8	8
6	2000	41	41	4	12	20	20	0	0	4	4
$b_j =$		80	96	20	36	44	48	10	18	22	24

TABLE IX
THE HIGH-PERFORMANCE SCHEMA STRUCTURES IN THE EXAMPLE

$\#(S_c)^a$	Schemata	Cost Value
1	(* * * * * 1)	2000
	(* * * 1 * *)	2400
	(* * * 0 * 1)	2000
2	(* 1 * * * 1)	2600
	(* * 1 * * 1)	3200
	(* * 1 0 * 1)	3200
3	(* * 1 * * 1)	3700
	(* 1 1 * * 1)	3800
	(* * 1 0 1 1)	3700
4	(* 1 1 0 * 1)	3800
	(0 * 1 0 1 1)	3700
	(* 1 1 0 0 1)	3800
5	(0 1 1 0 0 1)	3800
	(0 1 1 0 0 1)	3800

^a $\#(S_c)$ means the number of specific positions contained in the schema.

term of $\#(S_c)$, the number of specific positions contained in the schema, in increasing order of the given example.

Optimal solution = 3800, $x_i = 1$ ($i = 2, 3, 6$); all others = 0.

In order to identify the high-performance schemata, it is necessary to study the dynamics of the schema frequencies as the search progresses. By examining which schemata have above-average fitness values, we can foresee the population of subsequent generations due to the fact that these schemata are expected to proliferate. Then, the structure of the future populations can be estimated through schema frequency and fitness. Fig. 16 shows the growth of the high-performance schemata that have $\#(S_c) = 1, 2, 3, 4, 5$, and 6, indicated in Table IX, as the search progresses in term of generations. At the first generation, only the schemata with $\#(S_c) = 1$ and $\#(S_c) = 2$ have the

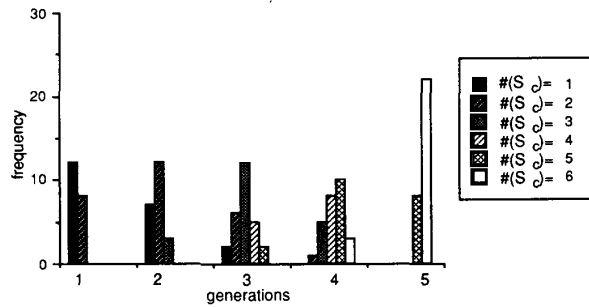


Fig. 16. Frequency histogram of the six schemata as the search progress.

frequency of occurrences 12 and 8, respectively. At the second generation, the frequency of schemata with $\#(S_c) = 1$ is decreased, the frequency of schemata with $\#(S_c) = 2$ is increased, and the schemata with $\#(S_c) = 3$ appear. While at the third generation the frequency of schemata with $\#(S_c) = 3$ become dominant in the population and the high performance with schema $\#(S_c) = 5$ appear. Eventually, at the fifth generation only the high-performance schemata, with $\#(S_c) = 5$ and with $\#(S_c) = 6$, exist in the population.

As we have stated in the previous paragraph, the concept of schemata restricts the search in a hyperplane. The underlying concept of schemata is that high-performance schemata are a degenerate description of the entire space that confines the search toward high-performance regions so that the search paths will eventually converge to an optimal or near-optimal point. Fig. 17 shows the percentage of reduction in the search space versus the number of specific positions contained in the schema. As $\#(S_c)$

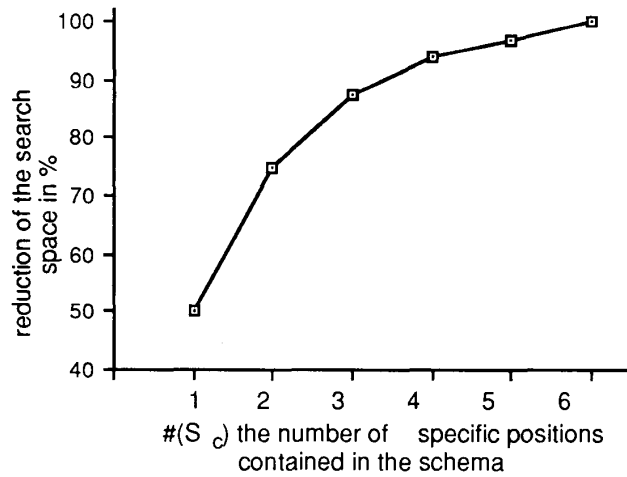


Fig. 17. The percentage of reduction in the search space of schemata.

$= 1$, the percentage of reduction in the search space is 50 percent. While $\#(S_c) = 6$, the percentage of reduction in the search space is 100 percent; this means that the search path is staying at the optimal point $X = [0, 1, 1, 0, 0, 1]$. The choice of the population size is essential to the quality of the final solution as well as the total running time required by the AG algorithm. According to the concept of schemata and the operations of genetic operators, the AG algorithm guarantees a better performance generation after generation. The larger population-size version may obtain a better performance population than that of the smaller population-size version, but at the expense of prolonging the execution time. Fig. 18 shows curves of the average cost of the population obtained by the AG algorithm in six generations with the population sizes of 6, 12, 18, and 24, respectively. The high-performance population appears in the larger population-size version. However, all the four versions with different population sizes converge to the optimal solution at the sixth generation. The larger population-size version needs more running time to converge than the smaller population-size ones. From our empirical results [15], the population size should depend on the structure of the underlying problem. Usually, the choice of population size is simply a constant factor of the problem size. A Markov chain theoretical analysis of the global convergence behavior and the running time complexity of the AG algorithm is in progress and will be reported in a future paper.

In fact, the proposed AG approach has been applied to other NP-hard optimization problems, e.g., the task assignment problem in distributed systems [16], and the one-dimensional bin-packing problem [15]. However, different problems may have to use different domain-specific knowledge. The representation scheme, the move generation strategy, and the genetic operators are needed to be specified differently to reflect the different structures of the problems.

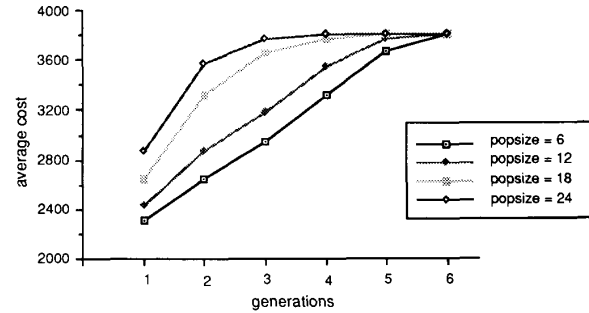


Fig. 18. The average cost of the population obtained by the AG in different population sizes.

B. The Representation Scheme

The solution representation of the underlying problem can be conceptually viewed as a point; in other words the feasible solutions of the problem are the points in the search space. For the MCKP and the set partitioning problems, the solution representation is quite straightforward and suitable for genetic operations. The solution is represented as a vector $X = [x_1, \dots, x_n]$ where $x_i = 1$ or $x_i = 0$, $1 \leq i \leq n$, and n is the problem size. The value of x_i is to denote whether an object i is included or not included in the current solution. However, the choice of an appropriate representation of the TSP suitable for genetic operators is a nontrivial task. In our representation, the cities are numbered from 1 to n . Each city i is denoted by its two coordinates (x_i, y_i) . The coordinates of these n cities are placed in an index table where the i th entry is the coordinate of city i . A tour is then represented by a vector X of size n containing the indexes of the table.

C. The Move Generation Strategy

There are two general strategies employed in AG as well as in SA for generating the next points of the search path in solving the MCKP and the set partitioning problems. The first is a greedy move and the second is a swapping move. A point in the search space is represented by a vector $X(1:n)$ where n is the problem size. The greedy move means, first, an element $X(i)$ is randomly selected from the current solution (point); second, the selected element is assigned to another value in an attempt to decrease the cost of the current solution and move to another point. For example, change $X(i)$ from 1 to 0. The swapping move means, we randomly choose two elements, say $X(i)$ and $X(j)$, which have been assigned to certain values, say 1 and 0, respectively, and then exchange their roles. That is, we exchange $X(i) = 1$ and $X(j) = 0$ to $X(i) = 0$ and $X(j) = 1$, respectively.

As for solving the TSP, only the swapping move strategy is used both in AG and in SA. The swapping strategies in the TSP algorithm are the random 2-exchange and the locally adjacent swap, which are shown in Fig. 19.

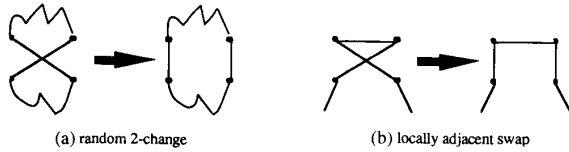


Fig. 19. The swapping strategy for the TSP.

D. The Genetic Operators

The three genetic operators not only modify the points in the old population to create a new population but also to decrease (or to increase) the average cost of the new population when the underlying problem is a minimization (or a maximization) problem. The crossover operator used in the algorithm is the usual crossover operator in the GA literature [8], [10]. Crossover takes two selected parents, e.g., two points in the old population, splits the structures at the same two randomly determined crossing sites, and then creates two offspring by swapping the middle portion of the structures. Hence, the function of the crossover is to generate rearrangements of coadapted groups of substructures from high-performance points. Nevertheless, crossover may produce the illegal structure of offspring. However, a penalty is added to the illegal structure. The second operator is inversion. Inversion alters the sequence between two randomly assigned indexes in a single point. This action modifies the structure of a single point, providing a more fertile ground for searching a better point by the greedy move in SA. The final operator is mutation. The intention of using the mutation operator in our approach is to ensure that all points in the search space remain reachable by SA. This can be easily achieved by changing a randomly selected element in a single point. However, mutation acts as a random search for a better point by our approach.

The genetic operators are performed according to the following steps.

Step 1: The parents are selected from the population based on their fitness values. Then, the crossover operator is applied to produce their offspring. The offspring must have their costs less than the average cost of the old generation, otherwise, the parents continue the following steps.

Step 2: The inversion operator is applied to the parents for reordering their own sequence to obtain new costs. If the new cost is lower than the old one, the parent is copied to the next generation; otherwise, the parents continue the next operation.

Step 3: The mutation operator is applied to the parents based on a predefined probability. Finally, the parents are copied to the next generation.

This cycle is repeated again and again until the next population of the next generation is generated.

Due to the fact that the solution representation in TSP is not a binary-string style, the genetic operators may con-

Step 1	Parent1 = [9 4 1 3 10 6 8 5 7 2] Child1 = [0 0 0 0 0 0 0 0 0 0]	Parent2 = [8 2 10 6 7 4 3 1 5 9] Child2 = [0 0 0 0 0 0 0 0 0 0]
Step 2	p1 = 3, p2 = 8 Parent1 = [9 4 1 3 10 6 8 5 7 2] Child1 = [# # 1 1 3 10 6 8 5 1 # #]	Parent2 = [8 2 10 6 7 4 3 1 5 9] Child2 = [# # 1 10 6 7 4 3 1 1 # #]
Step 3	Mating1 = [10 0 6 0 1 4 0 3 0 7]	Mating2 = [5 0 8 6 0 3 10 0 0 1]
Step 4	Child1 = [4 2 1 1 3 10 6 8 5 7 9]	Child2 = [9 8 1 10 6 7 4 3 1 5 2]

Fig. 20. An example of the crossover operation.

struct illegal tours. The penalty approach for infeasible solutions is not suitable for TSP. Therefore, the genetic operators must construct legal tours. Among the genetic operators used in our approach, the crossover is the most difficult operation to implement. Our crossover operation is one of the fundamental parts of the TSP algorithm and is described as follows:

- Step 1: Select two parents, Parent1 [1 : n] and Parent2 [1 : n], from the population based on their fitness values. Initially, Child1 [1 : n] := Child2 [1 : n] := 0.
- Step 2: Randomly drawing two indexes p_1, p_2 to serve as the crossover points. Then, Child1 [$p_1 : p_2$] := Parent1 [$p_1 : p_2$], and Child2 [$p_1 : p_2$] := Parent2 [$p_1 : p_2$].
- Step 3: Initialize two matching vectors and then setting their corresponding indexes. That is, Mating1 [1 : n] := Mating2 [1 : n] := 0. Let Mating1 [Parent1 [$p_1 : p_2$]] := Parent2 [$p_1 : p_2$], and Mating2 [Parent2 [$p_1 : p_2$]] := Parent1 [$p_1 : p_2$].
- Step 4.1: For each Child1 [i] = 0, $1 \leq i \leq n$, perform the following steps:
 $k := \text{Parent2}[i]$
while (Mating1 [k] $\neq$ 0) $k := \text{Mating1}[k]$;
Child1 [i] := k
- Step 4.2: For each Child2 [i] = 0, $1 \leq i \leq n$, perform the following steps:
 $k := \text{Parent1}[i]$
while (Mating2 [k] $\neq$ 0) $k := \text{Mating2}[k]$;
Child2 [i] := k

Fig. 20 shows an example of the crossover operation. After the crossover, the child whose cost is below the average cost of the population is to be copied to the new population.

VI. CONCLUSION

We have presented an annealing-genetic algorithm for solving three well-known NP-hard problems. The key features of this approach are the incorporation of the advantages of genetic algorithms into simulated annealing.

One of the major issues of many annealing schedules is the concept of quasi-equilibrium [1], [12]. The precise meaning of this concept is one of the key points differentiating one annealing schedule from the others. In our approach, the genetic operators are applied to the population that has been generated at each temperature by simulated annealing to induce maturation. Next, we impose the rule that the offspring must have their costs be less than the average cost of the last generation, otherwise, the parents are reproduced in this generation. This simple rule is intended to guarantee the convergence of the algorithm and also to make a shorter Markov chain that is generated at each temperature to reach the quasi-equilibrium.

Our results of an empirically $O(n^2)$ AG algorithm may have very important practical implications in solving the TSP. As far as we know, the best theoretical result for the regular-grid TSP of SA is $O(n^4 \log n)$ [12]. Even for heuristic methods, most of those are at best $O(n^3)$ [7], and the famous Lin-Kernighan heuristic is empirically $O(n^{2.2})$ [17]. In addition, the new best-known solution of 420 in Oliver's 30-city problem is obtained by use of our algorithm.

In conclusion, we would like to point out that our work has produced the following important results for improving SA and for solving some large scale NP-hard problems:

- 1) A new way for determining an adequate initial temperature value and its relevant sufficient initial state.
- 2) There are multiple search points in a population instead of a single point; the probability of trapping into local minima is greatly reduced.
- 3) The crossover of population for generating lower average cost generation denotes the quasi-equilibrium of the Markov chain at each temperature, such that the length of the Markov chain is bounded by the population size.
- 4) The frozen condition of the system is the convergence of the population that seems more accurate and easier to detect than SA.
- 5) Although SA is theoretically guaranteed to converge to global optimums with probability 1 if the Markov chains are of infinite length, in practice efficient annealing schedules are difficult to design. The genetic approach to SA, e.g., our proposed annealing-genetic algorithm, seems to facilitate the exhaustive and parallel treatment of the problem and to increase the probability of finding global minimums.
- 6) Empirical results reveal that our approach gives near-optimal solutions with the gaps to the optimal solutions less than 0.03 percent for the set partitioning problem, less than 1 percent for the MCKP, and around 3 percent for the TSP, all in a reasonable computation time.
- 7) The AG algorithm for these problems is empirically an $O(n^2)$ algorithm.

ACKNOWLEDGMENT

The authors would like to thank the referees for providing many useful comments and suggestions on the previous draft. The authors also thank the anonymous referee for bringing the paper by Davis and Ritter [3] to their attention.

REFERENCES

- [1] E. Aarts and J. Korst, *Simulated Annealing and Boltzmann Machines—A Stochastic Approach to Combinatorial Optimization and Neural Computing*. New York: Wiley, 1989.
- [2] L. Davis, *Genetic Algorithms and Simulated Annealing*. Los Altos, CA: Morgan Kaufmann, 1987.
- [3] L. Davis and F. Ritter, "Schedule optimization with probabilistic search," in *Proc. 3rd IEEE Conf. Artif. Intellig. Applicat.*, 1987, pp. 231-236.
- [4] M. Garey and D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: W. H. Freeman & Company, 1979.
- [5] S. Geman and D. Geman, "Stochastic relaxation, Gibbs distribution, and the Bayesian restoration of images," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 6, pp. 721-741, 1984.
- [6] D. E. Goldberg, *Genetic Algorithms: In Search, Optimization and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
- [7] B. L. Golden, L. D. Bodin, T. Doyle, and W. Stewart, "Approximate traveling salesman problem," *Operation Res.*, vol. 28, pp. 694-711, 1980.
- [8] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: Univ. Michigan Press, 1975.
- [9] M. D. Huang, F. Romeo, and A. Sangiovanni-Vincentelli, "An efficient general cooling schedule for simulated annealing," in *Proc. IEEE Int. Conf. Computer-Aided Design*, Santa Clara, CA, 1986, pp. 381-384.
- [10] A. Kenneth, De Jong, and W. M. Spears, "Using genetic algorithms to solve NP-Complete problems," in *Proc. 3rd Int. Conf. Genetic Algorithms*, George Mason University, 1989, pp. 124-132.
- [11] S. Kirkpatrick, C. D. Gellatt Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671-680, 1983.
- [12] P. J. M. van Laarhoven and E. H. L. Aarts, *Simulated Annealing: Theory and Applications*. Dordrecht, Holland: D. Reidel, 1987.
- [13] F. T. Lin and C. C. Hsu, "Task assignment problems in distributed computing systems by simulated annealing," *J. Chinese Instit. Engineers*, vol. 14, no. 5, pp. 537-550, 1991.
- [14] F. T. Lin, C. C. Hsu, and C. Y. Kao, "An efficient annealing schedule for multiconstraint zero-one knapsack problem," *Int. Comput. Symp. 1990*, Hsinchu, Taiwan, pp. 309-314, 1990.
- [15] F. T. Lin and C. Y. Kao, "A stochastic approach for the one-dimensional bin-packing problem," in *Proc. 1992 IEEE Int. Conf. Syst. Man, Cybern.*, Chicago, Illinois, Oct. 18-21, 1992, to be published.
- [16] F. T. Lin and C. Y. Kao, "Using the concepts of genetic algorithms to design the annealing schedule for solving the task scheduling problem," in *Proc. 5th Int. Symp. Artif. Intell.*, Cancun, Mexico, Dec. 7-11, 1992, to be published.
- [17] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling salesman problem," *Operation Res.*, vol. 21, pp. 498-516, 1973.
- [18] D. G. Luenberger, *Linear and Nonlinear Programming*. Reading, MA: Addison-Wesley, 1984, 2nd ed.
- [19] C. H. Papadimitriou and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*. Englewood Cliffs, NJ: Prentice-Hall, 1982.
- [20] C. C. Peterson, "Computational experience with variants of the Balas algorithm applied to the selection of R & D projects," *Management Sci.*, vol. 13, pp. 736-750, 1967.
- [21] G. Sasaki and B. Hajek, "The time complexity of maximum matching by simulated annealing," *J. Assoc. Comput. Mach.*, vol. 35, no. 2, pp. 387-403, 1988.
- [22] S. Senju and Y. Toyoda, "An approach to linear programming with 0-1 variables," *Management Sci.*, vol. 15, pp. 196-207, 1968.

- [23] S. R. White, "Concepts of scale in simulated annealing," in *Proc. IEEE Int. Conf. Comput. Design*, Port Chester, 1984, pp. 645-651.
- [24] D. Whitley, T. Starkweather, and D. Fuquay, "Scheduling problems and traveling salesman: The genetic edge recombination operator," in *Proc. 3rd Int. Conf. Genetic Algorithms*, George Mason University, 1989, pp. 133-140.

Feng-Tse Lin was born in Taipei, Taiwan, on September 26, 1951. He received the B.S. degree in computer science from Tamkang University, Tamsui, Taiwan, in 1975, the M.S. degree in computer engineering from National Chiao Tung University, Hsichu, Taiwan, in 1984, and the Ph.D. degree in computer science and information engineering from National Taiwan University, Taipei, Taiwan, in 1991.

From 1984 to 1987, he was an Assistant Researcher of Computer Technology at Telecommunication Laboratories, Chung Li, Taiwan. Currently, he is an Associate Professor with the Department of Applied Mathematics, Chinese Culture University, Yangminshan, Taipei, Taiwan. His current research interests include genetic algorithms, simulated annealing, and combinatorial optimization problems.

Dr. Lin is a member of the IEEE Computer Society and the Association for Computing Machinery.

Cheng-Yan Kao was born in Taipei, Taiwan, on October 3, 1948. He received the B.S. degree in mathematics from National Taiwan University, Taipei, Taiwan, in 1971, and the M.S. degree in computer science in 1976, the M.S. degree in statistics in 1978, and the Ph.D. degree in computer science in 1981, all from the University of Wisconsin-Madison.

He has previously been with Ford Aerospace, the Unisys Corporation, and was with General Electric from 1980 to 1989 at the Johnson Space Center, NASA, Houston, TX. He has been a professor with the Department of Computer Science and Information Engineering, National Taiwan University since 1989. He has published more than 20 technical papers in various journals and conference records. His research interests include genetic algorithms, simulated annealing, scheduling theory, mathematical programming, artificial intelligence, and software engineering.

Dr. Kao is a member of ORSA and the AAAI.

Ching-Chi Hsu was born in Taipei, Taiwan, on January 28, 1949. He received the B.S. degree in physics from National Tsing Hua University in 1971, Hsichu, Taiwan, and both the M.S. and Ph.D. degrees in computer engineering from National Taiwan University, Taipei, Taiwan, in 1975 and 1982, respectively.

In 1977, he joined the faculty of the Department of Computer Science and Information Engineering at National Taiwan University as a Lecturer, where he is currently a Professor and the Director of Artificial Intelligence Laboratory. His current research interests include computer architecture and language for AI, learning and knowledge acquisition, and distributed processing of knowledge.